

WikiRMI — mowa do nauczenia na pamięć + pytania na obronę

Zespół: Maciej Wawer, Krystian Wasil, Marek Więcaszek · ~10 min · nacisk na wielowątkowość

OSOBA 1 — Maciej Wawer (~3 min)

Dzień dobry. Chcielibyśmy przedstawić nasz projekt — **WikiRMI**, czyli rozproszony system typu wiki. Wybraliśmy temat dziewiąty: administrator tworzy szkielety stron oraz konta użytkowników, a użytkownicy mogą te strony przeglądać i modyfikować. Całość działa jako aplikacja **rozproszona** — klient i serwer to osobne programy, które komunikują się przez sieć za pomocą mechanizmu **RMI**, czyli zdalnego wywoływania metod.

Zacznę od architektury. Projekt dzieli się na trzy warstwy: wspólny kontrakt, serwer i klienta. Kontrakt to interfejs zdalny — w Javie dziedziczy on po interfejsie Remote, a każda jego metoda może zgłosić wyjątek RemoteException. Serwer dostarcza implementację tego interfejsu; nasza klasa serwera dziedziczy po UnicastRemoteObject, dzięki czemu staje się **obiektem zdalnym**, dostępnym przez sieć.

Jak to się spina? Serwer tworzy **rejestr RMI** i rejestruje w nim swój obiekt poleceniem Naming.rebind, pod adresem w formacie „lokalizator, host, dwukropek, port, nazwa”. Klient pobiera z rejestru **namiastkę**, czyli stub, poleceniem Naming.lookup, i od tej pory wywołuje metody serwera tak, jakby były lokalne — choć w rzeczywistości wykonują się na serwerze. RMI działa u nas w obie strony: serwer również wywołuje metody po stronie klienta, czyli tak zwane callbacki, żeby na bieżąco wysyłać powiadomienia o zmianach.

I tu dochodzimy do sedna. **RMI obsługuje każde wywołanie klienta w osobnym wątku** po stronie serwera. To znaczy, że gdy kilku użytkowników działa jednocześnie, wiele wątków serwera sięga po te same dane — na przykład po tę samą stronę. Bez synchronizacji powstałby **stan wyścigu**, a jeden zapis mógłby nadpisać drugi. Jak rozwiązaliśmy ten problem — opowie teraz Krystian.

OSOBA 2 — Krystian Wasil (~4 min) — część najważniejsza

Dziękuję. Najważniejsza w naszym projekcie jest poprawna **wielowątkowość**, więc poświęcę jej najwięcej czasu. Główna sekcja krytyczna to sytuacja, w której **dwóch użytkowników edytuje tę samą stronę w tym samym momencie**. Gdybyśmy tego nie kontrolowali, doszłoby do utraty zapisu albo odczytu strony w stanie niespójnym. Cały rdzeń współbieżności umieściliśmy świadomie w **jednym pliku — klasie WikiStore** — i użyliśmy w nim **czterech klasycznych mechanizmów**, każdy w osobnej, opisanej sekcji.

Pierwszy mechanizm to blokady — ReentrantReadWriteLock. Każda strona ma własną blokadę odczytu i zapisu. To blokowanie drobnziarniste — blokujemy pojedynczą stronę, a nie cały serwer. Blokada odczytu pozwala wielu użytkownikom czytać tę samą stronę jednocześnie, a blokada zapisu jest wyłączna — tylko jeden wątek może zapisywać. Tu jest serce ochrony przed wyścigiem: pozyskanie blokady edycji działa metodą „sprawdź i ustaw” pod blokadą zapisu. Dzięki temu, gdy dziesięć wątków równocześnie spróbuje wejść w edycję, **dokładnie jeden** zakłada blokadę, a pozostali dostają wyjątek PageLockedException.

Drugi mechanizm to monitory — słowo kluczowe synchronized. Używamy go do wspólnego licznika zapisów. To klasyczny monitor: blok synchronized wpuszcza tylko jeden wątek naraz. Gdyby go nie było, zwykłe „licznik plus plus” gubiłoby zliczenia, bo operacja odczyt–dodaj–zapis nie jest niepodzielna.

Trzeci mechanizm to semaforey — klasa Semaphore. Ogranicza liczbę jednocześnie zalogowanych klientów. Logowanie pobiera jedno pozwolenie metodą tryAcquire, a wylogowanie je oddaje metodą

release. Gdy pozwoleń braknie — serwer odmawia kolejnego logowania.

Czwarty mechanizm to wątki. Mamy wątek demon, który w tle cyklicznie zwalnia przeterminowane blokady — na przykład gdy klient się zawiesił. Mamy też pulę wątków, która wysyła powiadomienia do klientów poza wątkiem edycji, żeby jeden wolny klient nie blokował pozostałych.

Na koniec dwa argumenty. Po pierwsze — **brak zakleszczeń**: każda operacja trzyma najwyżej jedną blokadę strony naraz i nie zagnieżdża blokad, więc cykl oczekiwania nie może powstać. Po drugie — **brak utraconych zapisów**: dodatkowo stosujemy optymistyczną kontrolę wersji. Pokażemy teraz, że to działa — Marek.

OSOBA 3 — Marek Więcaszek (~3 min)

Dziękuję. Przejdźmy do **demonstracji na żywo**. Mamy uruchomione dwa okna klienta, oba zalogowane na to samo konto. W pierwszym oknie wybieram stronę i klikam Edytuj — od tej chwili ten klient trzyma blokadę. Teraz w drugim oknie próbuję otworzyć tę samą stronę do edycji — i dostaję komunikat: „Strona jest aktualnie edytowana przez admina”. To jest **stan wyścigu zablokowany na żywo**. Teraz w pierwszym oknie zapisuję zmianę — i lista w drugim oknie **odświeżyła się sama**. To zadziałał callback RMI, czyli powiadomienie z serwera do klienta.

Tę samą poprawność potwierdzają **testy współbieżności**. Najważniejszy uruchamia **dziesięć wątków**, które jednocześnie próbują zająć tę samą stronę — i za każdym razem jeden zdobywa blokadę, a dziewięć zostaje odrzuconych. Drugi test uruchamia **pięćdziesięciu** edytorów po kolei i sprawdza, że numer wersji końcowej to dokładnie pięćdziesiąt — czyli żaden zapis nie ginie. Łącznie mamy **siedemnaście testów i wszystkie przechodzą**, w tym test integracyjny sprawdzający wyścig „przez sieć”, po RMI.

Na koniec dane i trwałość. Stan zapisujemy do czytelного pliku JSON, atomowo, więc przerwany zapis nie uszkodzi danych. Po **restarcie serwera** wszystkie strony, użytkownicy i historia są w pełni odtwarzane, a liczbę jednoczesnych klientów pilnuje semafor.

Podsumowując: zbudowaliśmy kompletny, działający system **rozproszony w RMI** z poprawną, drobnoziarnistą i **wolną od zakleszczeń wielowątkowością**, pokazaną na czterech klasycznych mechanizmach. Dziękujemy za uwagę i zapraszamy do pytań.

Fiszki — możliwe pytania komisji i krótkie odpowiedzi

P: Czym różni się monitor (synchronized) od blokady (Lock)?

O: Monitor (synchronized) jest wbudowany w każdy obiekt, prosty, ale 'wszystko albo nic' — zwalnia się automatycznie na końcu bloku. Lock (np. ReentrantLock, ReentrantReadWriteLock) jest jawny, daje więcej: rozdział odczytu/zapisu, tryLock z czasem, lockInterruptibly.

P: Dlaczego ReentrantReadWriteLock zamiast zwykłego synchronized?

O: Bo aplikacja jest 'dużo odczytów, mało zapisów'. ReadWriteLock pozwala wielu czytelnikom czytać RÓWNOCZEŚNIE, a synchronized serializowałby każdy odczyt — gorsza przepustowość.

P: Co to jest stan wyścigu (race condition) i jak go zapobiegacie?

O: To sytuacja, gdy wynik zależy od kolejności przepływu wątków. Zapobiegamy sekcją krytyczną 'sprawdź-i-ustaw' blokady edycji pod writeLock — z N wątków dokładnie jeden zakłada blokadę.

P: Dlaczego nie ma zakleszczeń (deadlock)?

O: Bo każda operacja trzyma najwyżej JEDNĄ blokadę strony naraz i nigdy nie zagnieżdża blokad — nie powstaje cykl oczekiwania, a bez cyklu deadlock jest niemożliwy.

P: Czym jest semafor i do czego go użyłście?

O: Semafor to licznik pozwoleń. Użyliśmy go do limitu jednoczesnych klientów: logowanie pobiera pozwolenie (tryAcquire), wylogowanie oddaje (release); brak pozwoleń = odmowa logowania.

P: Jak RMI radzi sobie z wielowątkowością?

O: Serwer RMI obsługuje każde przychodzące wywołanie w OSOBNYM wątku z puli. Dlatego implementacja musi być bezpieczna wątkowo — i to my zapewniamy w WikiStore.

P: Co to UnicastRemoteObject, stub i rejestr RMI?

O: UnicastRemoteObject to baza obiektu zdalnego (eksport przez sieć). Stub to lokalna 'namiastka' obiektu zdalnego u klienta. Rejestr RMI to katalog nazwa→obiekt, w którym serwer rejestruje usługę (Naming.rebind), a klient ją odnajduje (Naming.lookup).

P: Czym są callbacki w RMI?

O: To wywołania w drugą stronę: serwer wywołuje metodę po stronie klienta. Klient implementuje drugi interfejs zdalny (WikiClientCallback) i rejestruje się; serwer powiadamia go o zmianach (np. że strona została zapisana).

P: Co to lost update i jak go unikacie?

O: Lost update = jedna aktualizacja nadpisuje drugą i ginie. Unikamy podwójnie: pesymistycznie (tylko posiadacz blokady zapisuje) i optymistycznie (zgodność numeru wersji). Inkrementacja wersji + dopis do historii są niepodzielne.

P: Po co wątek daemon i czym różni się od zwykłego?

O: Wątek demon sprząta w tle przeterminowane blokady (np. po awarii klienta). Różnica: wątek demon NIE wstrzymuje zamknięcia JVM — gdy zostaną tylko demony, program się kończy.

P: Po co volatile i ConcurrentHashMap?

O: volatile zapewnia bezpieczną WIDOCZNOŚĆ pola między wątkami (tu: czas ostatniej aktywności sesji). ConcurrentHashMap to mapa bezpieczna wątkowo; putIfAbsent daje atomowe 'utwórz, jeśli nie istnieje' bez globalnej blokady.

P: Jaka jest maksymalna liczba jednoczesnych klientów i jak reaguje system na restart?

O: Limit pilnuje semafor — domyślnie 50, konfigurowalne. Po restarcie serwer wczytuje stan z pliku wiki.json (zapis atomowy), więc strony, konta i historia są w pełni odtwarzane; blokady są ulotne i nie są utrwalane.